

CSCI544 Homework 3: Named Entity Recognition (NER)

Name: Rudrani Chavarkar

Course: CSCI 544 – Applied Natural Language Processing

USC ID: 3158210765

1. Introduction

In this assignment, we implemented deep learning models for **Named Entity Recognition (NER)** using the CoNLL-2003 dataset. The objective is to assign each token in a sentence a label using the **BIO tagging scheme**, where:

- **B-** indicates the beginning of an entity
- **I-** indicates continuation of the entity
- **O** indicates non-entity tokens

We developed three models of increasing complexity:

1. **Task 1:** Baseline Bidirectional LSTM (BiLSTM)
2. **Task 2:** BiLSTM with pretrained GloVe embeddings and capitalization features
3. **Bonus:** BiLSTM with character-level CNN

This assignment required both implementation and experimentation with different modeling strategies.

2. Dataset Description

The dataset consists of three splits:

- Training set
- Development (dev) set
- Test set

Each line in the dataset contains: index word tag

Sentences are separated by blank lines, and the tags follow the BIO format.

3. Task 1: Simple Bidirectional LSTM Model

3.1 Model Architecture

The architecture used is: **Embedding** → **BiLSTM** → **Linear** → **ELU** → **Classifier**

3.2 Implementation Details

Data Processing

- Built a vocabulary (word2idx) from training data

- Converted words into indices
- Used <PAD> and <UNK> tokens
- Applied padding within batches

To efficiently process sequences of different lengths, **packed sequences** were used before feeding data into the LSTM.

Model Components

1. **Embedding Layer**
 - Dimension: 100
 - Randomly initialized and trained
2. **BiLSTM Layer**
 - Hidden size: 256
 - Bidirectional → captures both past and future context
3. **Linear + ELU**
 - Reduces dimensionality to 128
 - ELU improves gradient flow
4. **Classifier**
 - Outputs tag probabilities

Training Strategy

- Loss: CrossEntropyLoss (ignoring padding)
- Optimizer: SGD with momentum
- Gradient clipping applied
- Learning rate scheduling used

3.3 Results (Dev Set)

Metric	Score
Precision	0.8630
Recall	0.7253
F1 Score	0.7882

Detailed evaluation:

- Accuracy: **95.49%**
- Precision: **86.30%**
- Recall: **72.53%**
- F1 Score: **78.82%**

Per-Entity Performance

Entity	Precision	Recall	F1
LOC	93.00%	80.95%	86.55

MISC	87.35%	77.87%	82.34
ORG	83.47%	66.67%	74.13
PER	80.63%	65.74%	72.43

3.4 Analysis

- High precision but lower recall → model is conservative
- Struggles more with ORG and PER entities
- Limited by randomly initialized embeddings

4. Task 2: GloVe Embeddings + Capitalization

4.1 Motivation

GloVe provides pretrained semantic knowledge, but it is **case-insensitive**, while NER depends heavily on capitalization.

4.2 Implementation Details

GloVe Embeddings

- Loaded pretrained 100D vectors
- Used lowercase fallback if word not found
- Random initialization for missing words
- Embeddings are fine-tuned during training

Capitalization Features

Each word is categorized into:

- lowercase
- uppercase
- title case
- mixed case
- contains digits

A **case embedding (dim=32)** is added and concatenated with word embeddings.

Input Representation:

Final input to LSTM: [word embedding] + [case embedding]

Training Improvements:

- Optimizer: AdamW
- Larger batch size (32)
- Different learning rates:

- Smaller for embeddings
- Larger for new layers
- Early stopping used

BIO Tag Repair:

Invalid sequences are corrected before evaluation to ensure consistency.

4.3 Results (Dev Set)

Metric	Score
Precision	0.9056
Recall	0.9106
F1 Score	0.9081

Detailed evaluation:

- Accuracy: **98.39%**
- Precision: **90.66%**
- Recall: **91.03%**
- F1 Score: **90.85%**

Per-Entity Performance

Entity	Precision	Recall	F1
LOC	95.06%	93.14%	94.09
MISC	87.03%	85.90%	86.46
ORG	83.94%	88.89%	86.35
PER	93.36%	93.05%	93.20

4.4 Analysis

- Significant improvement over Task 1
- Balanced precision and recall
- Capitalization greatly improves PER and LOC detection
- GloVe improves generalization

5. Bonus: Character-Level CNN Model

5.1 Implementation Details

Character Representation

- Built character vocabulary
- Converted words into character sequences

Character Embedding

- Each character mapped to vector (dim=30)

CNN Module

- Multiple filters (e.g., 3, 4, 5)
- Captures prefixes/suffixes
- Max pooling produces fixed-size vector

Final Representation

[word embedding] + [case embedding] + [char CNN features]

5.2 Results (Dev Set)

Metric	Score
Precision	0.8685
Recall	0.8790
F1 Score	0.8737

5.3 Analysis

- Improves handling of rare words
- Slightly worse than Task 2 due to:
 - Increased complexity
 - Need for further tuning

6. Model Comparison

Model	Precision	Recall	F1 Score
Task 1	0.8630	0.7253	0.7882
Task 2	0.9056	0.9106	0.9081
Bonus	0.8685	0.8790	0.8737

7. Key Insights

- Pretrained embeddings significantly improve performance
- Capitalization is critical for NER

- Character features help but require tuning
- Best performance achieved in Task 2

8. Challenges

- Hyperparameter tuning required multiple experiments
- Training time managed using batching
- Case sensitivity handled via embeddings
- BIO inconsistencies fixed using repair function

9. Conclusion

This assignment demonstrates the effectiveness of combining:

- Contextual modeling (BiLSTM)
- Pretrained embeddings (GloVe)
- Linguistic features (capitalization, characters)

The final model achieves an F1 score of **0.9081**, indicating strong performance on the NER task.